Exercício Prático 06

Aluno: Thiago Henrique Gomes Feliciano - 1543790

Parte 1 - Responda

1. O que é um arquivo fonte?

A. um arquivo de texto que contém instruções de linguagem de programação.

2. O que é um registrador?

B. uma parte do processador que possui um padrão de bits.

3. Qual o caracter que, na linguagem assembly do SPIM, inicia um comentário?

A. #.

4. Quantos bits há em cada instrução de máquina MIPS?

C. 32.

5. O que é o contador de programa?

D. parte do processador que contém o endereço da próxima instrução de máquina para ser obtida.

6. Ao executarmos uma instrução, quanto será adicionado ao contador de programa?

C. 4.

7. O que é uma diretiva, tal como a diretiva .text?

D. uma declaração que diz o montador algo sobre o que o programador quer, mas não corresponde diretamente a uma instrução de máquina.

8. O que é um endereço simbólico?

D. um nome usado no código-fonte em linguagem assembly para um local na memória.

9. Em qual endereço o simulador SPIM coloca a primeira instrução de máquina quando ele está sendo executado?

B. 0x00400000.

10. Algumas instruções de máquina possuem uma constante como um dos operandos. Como é chamado tal operando?

A. operando imediato.

11. Como é chamada uma operação lógica executada entre bits de cada coluna dos operandos para produzir um bit de resultado para cada coluna?

B. operação bitwise.

12. Quando uma operação é de fato executada, como estão os operandos na ALU?

D. Cada um dos registradores deve possuir 32 bit.

13. Dezesseis bits de dados de uma instrução de ori são usados como um operando imediato. Durante execução, o que deve ser feito primeiro?

B. Os dados são estendidos em zero à esquerda por 16 bits.

14. Qual das instruções seguintes armazenam no registrador $5 um padrão de bits que representa positivo 48?

A. ori $5,$0,0x48.

15. A instrução de ori pode armazenar o complemento de dois de um número em um registrador?

A. Não.

16. Qual das instruções seguintes limpa todos os bits no registrador $8 com exceção do byte de baixa ordem que fica inalterado?

D. andi $8,$8,0xFF

17. Qual é o resultado de um ou exclusivo de padrão sobre ele mesmo?

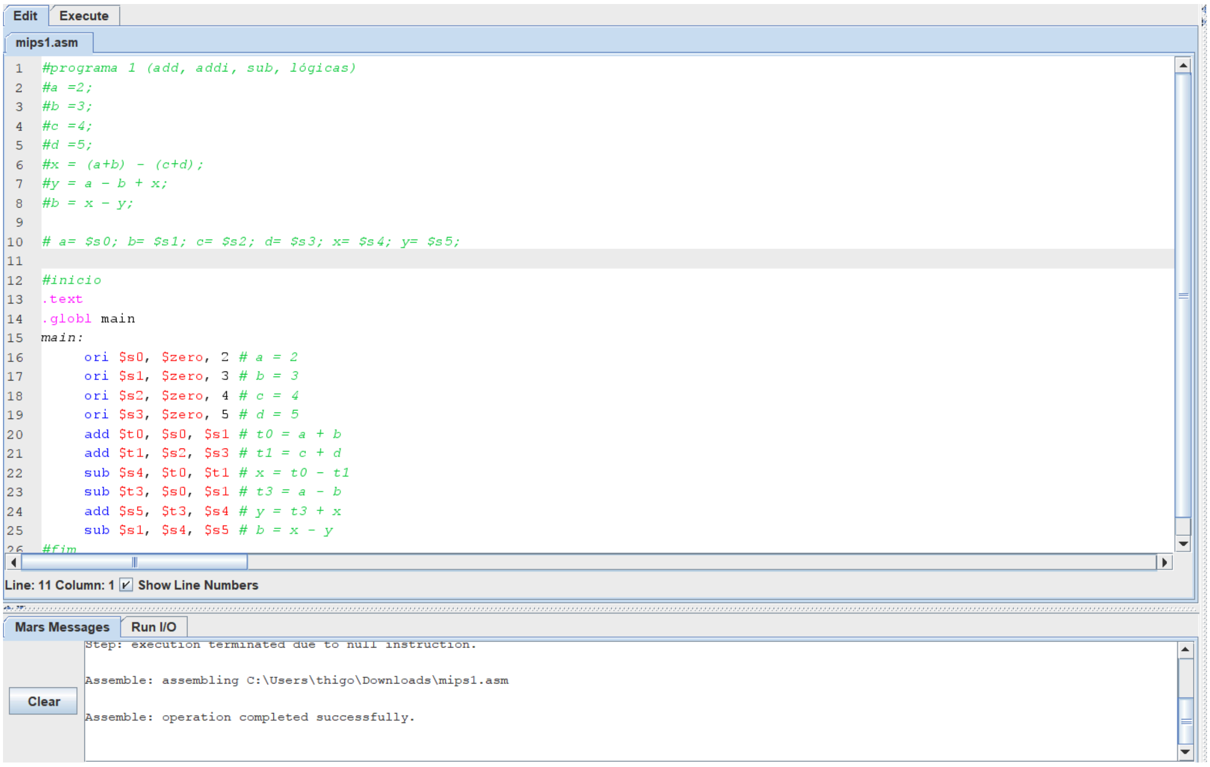
A. Todos os bits em zero.

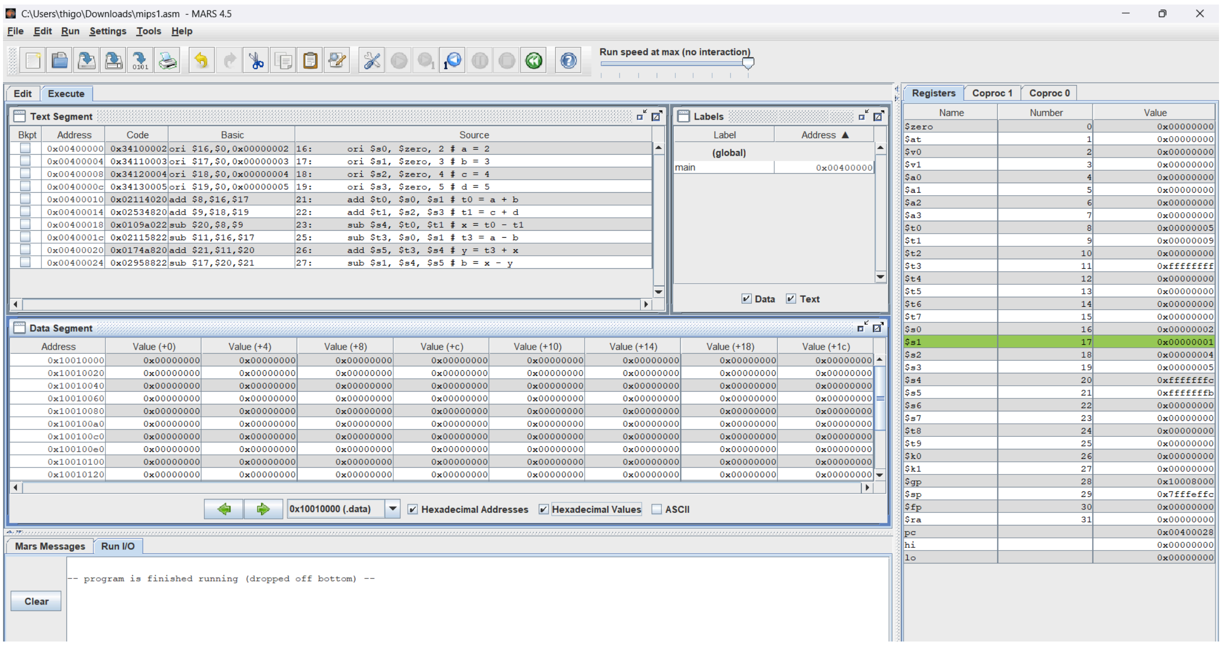
18. Todas as instruções de máquina têm os mesmos campos?

A. Não. Diferentes de instruções de máquina possuem campos diferentes.

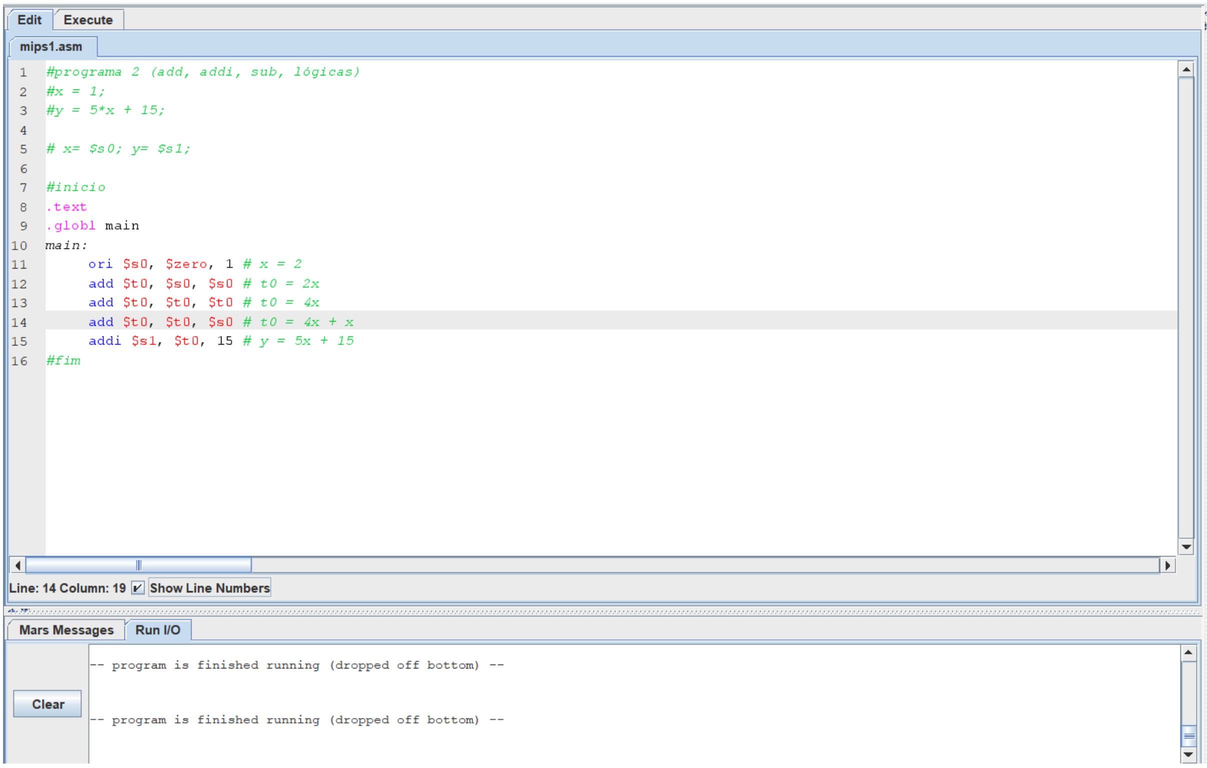
Parte 2 - Implementar em MIPS/MARS os seguintes programas (usando apenas as instruções indicadas)

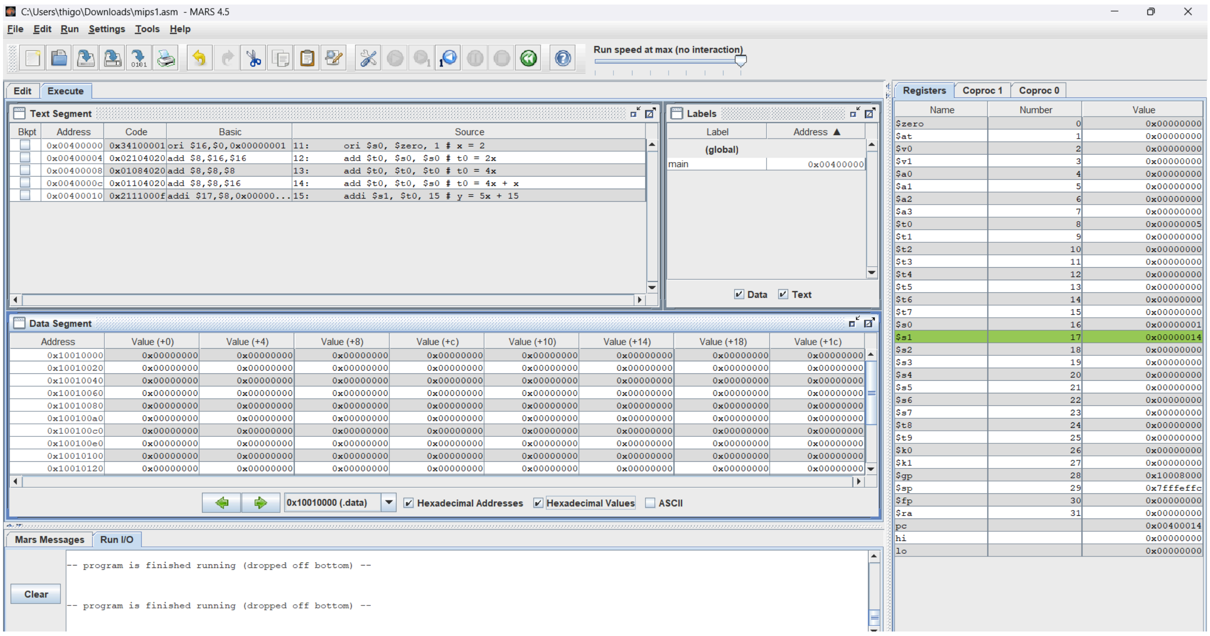
Programa 1:





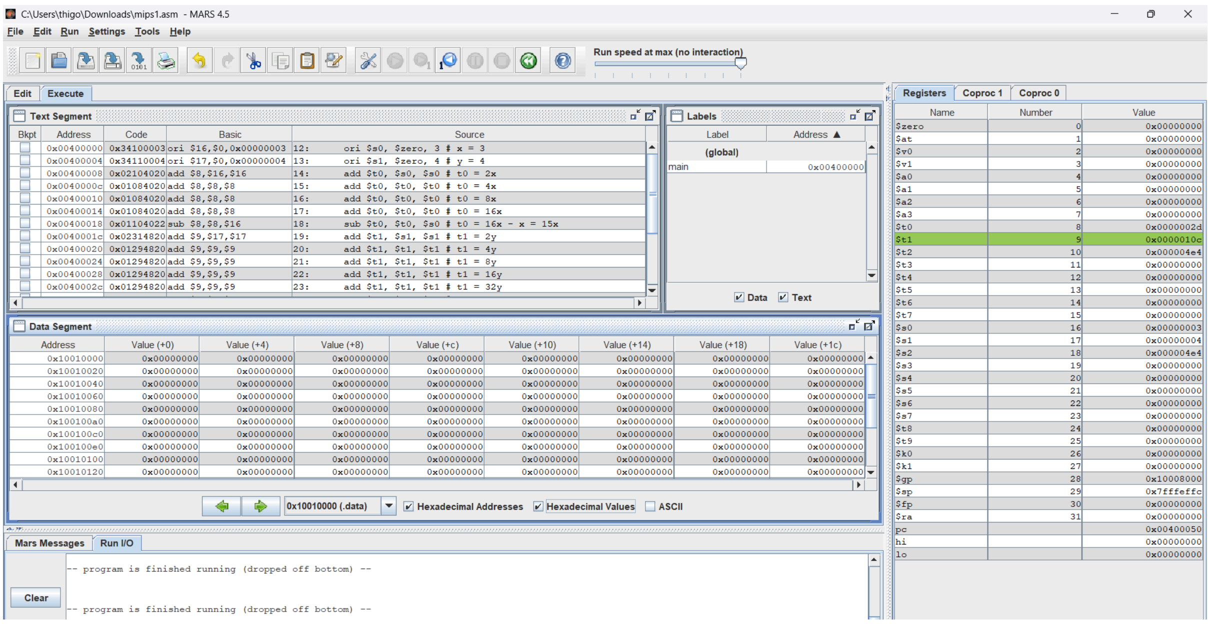
Programa 2:


Edit
Execute
mips1.asm
#programa 2 (add, addi, sub, lógicas)
#x = 1;
#y = 5*x + 15;
x= $s0; y= $s1;
#inicio
.text
.globl main
main:
ori $s0, $zero, 1 # x = 2
add $t0, $s0, $s0 # t0 = 2x
add $t0, $t0, $t0 # t0 = 4x
add $t0, $t0, $s0 # t0 = 4x + x
addi $s1, $t0, 15 # y = 5x + 15
#fim
Line: 14 Column: 19
Show Line Numbers
Mars Messages
Run I/O
Clear
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --


C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit
Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x34100001 ori $16,$0,0x00000001 11: ori $s0, $zero, 1 # x = 2
0x00400004 0x02104020 add $8,$16,$16 12: add $t0, $s0, $s0 # t0 = 2x
0x00400008 0x01084020 add $8,$8,$8 13: add $t0, $t0, $t0 # t0 = 4x
0x0040000c 0x01104020 add $8,$8,$16 14: add $t0, $t0, $s0 # t0 = 4x + x
0x00400010 0x2111000f addi $17,$8,0x00000... 15: addi $s1, $t0, 15 # y = 5x + 15
Labels
Label Address
(global)
main 0x00400000
Data
Text
Data Segment
Hexadecimal Addresses
Hexadecimal Values
ASCII
Registers
Coproc 1
Coproc 0
Name Number Value
$t0 8 0x00000005
$s0 16 0x00000001
$s1 17 0x00000014
$gp 28 0x10008000
$sp 29 0x7fffeffc
pc 0x00400014
Mars Messages
Run I/O
Clear
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --

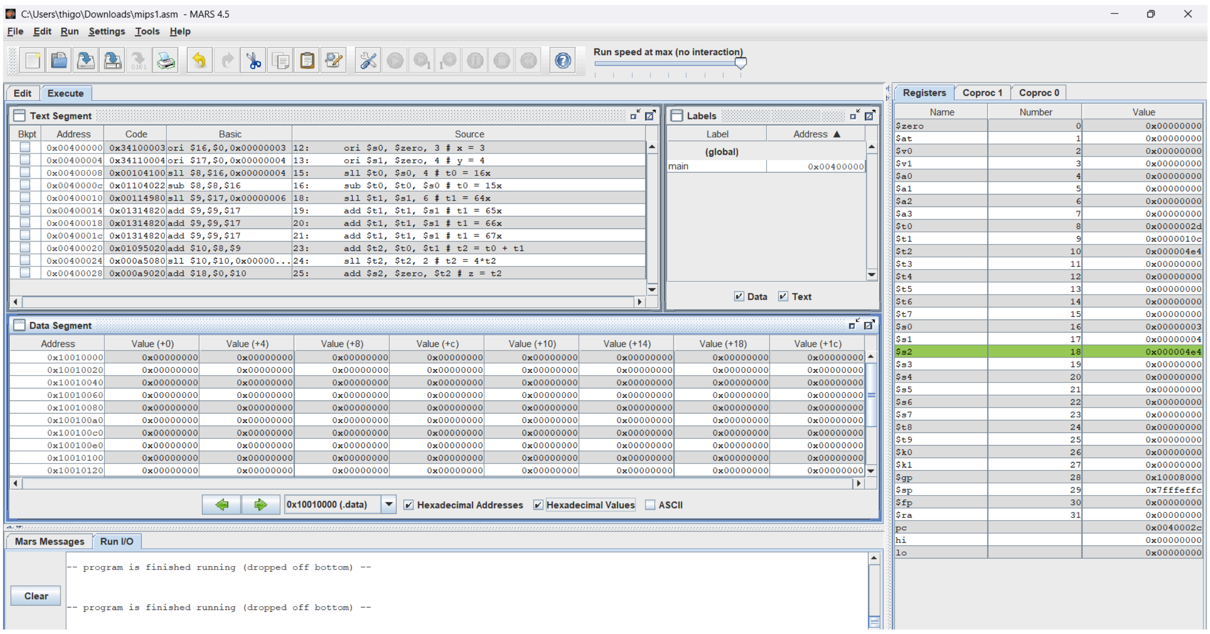
Programa 3:

```
#programa 3 (add, addi, sub, lógicas)
#x = 3;
#y = 4 ;
#z = ( 15*x + 67*y)*4
# x= $s0; y= $s1; z= $s2;

#inicio
.text
.globl main
main:
	ori $s0, $zero, 3 # x = 3
	ori $s1, $zero, 4 # y = 4
	add $t0, $s0, $s0 # t0 = 2x
	add $t0, $t0, $t0 # t0 = 4x
	add $t0, $t0, $t0 # t0 = 8x
	add $t0, $t0, $t0 # t0 = 16x
	sub $t0, $t0, $s0 # t0 = 16x - x = 15x
	add $t1, $s1, $s1 # t1 = 2y
	add $t1, $t1, $t1 # t1 = 4y
	add $t1, $t1, $t1 # t1 = 8y
	add $t1, $t1, $t1 # t1 = 16y
	add $t1, $t1, $t1 # t1 = 32y
	add $t1, $t1, $t1 # t1 = 64y
	add $t1, $t1, $s1 # t1 = 65y
	add $t1, $t1, $s1 # t1 = 66y
	add $t1, $t1, $s1 # t1 = 67y
	add $t2, $t0, $t1 # t2 = t0 + t1
	add $t2, $t2, $t2 # t2 = 2*t2
	add $t2, $t2, $t2 # t2 = 4*t2
	add $s2, $zero, $t2 # z = (15x + 67y)* 4
#fim
```

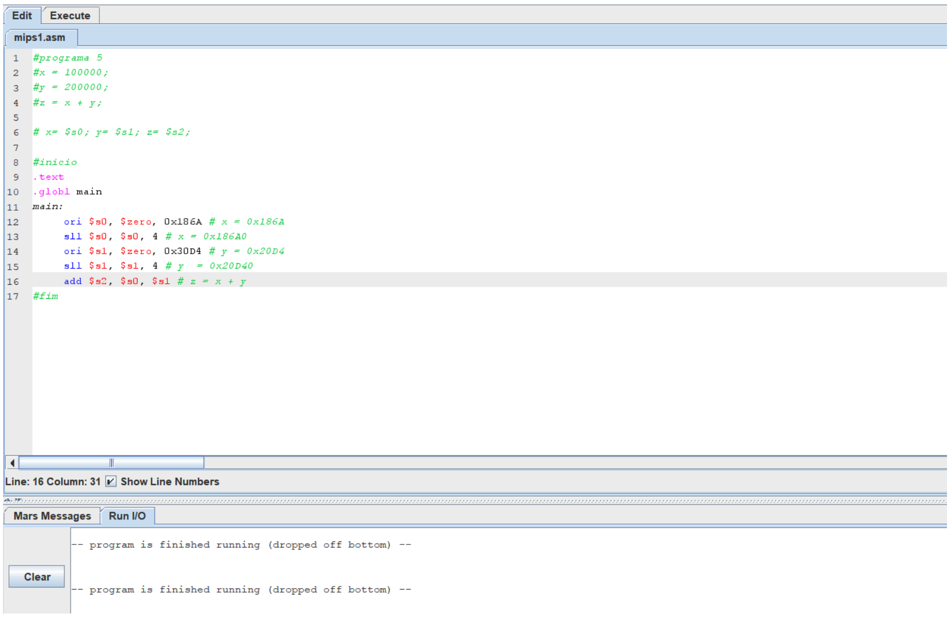


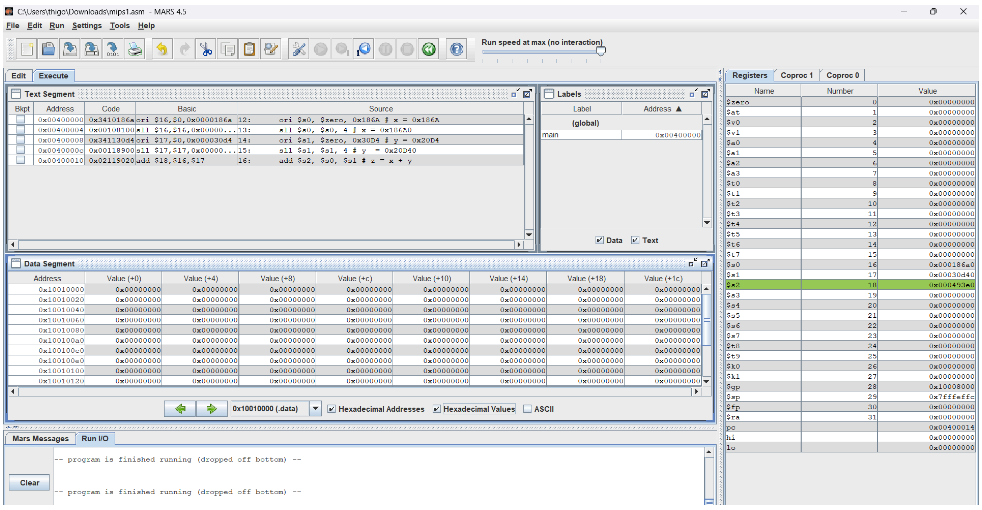
Programa 4:

```
#programa 4
#x = 3;
#y = 4 ;
#z = ( 15*x + 67*y)*4

# x= $s0; y= $s1; z= $s2;

#inicio
.text
.globl main
main:
	ori $s0, $zero, 3 # x = 3
	ori $s1, $zero, 4 # y = 4

	sll $t0, $s0, 4 # t0 = 16x
	sub $t0, $t0, $s0 # t0 = 15x

	sll $t1, $s1, 6 # t1 = 64x
	add $t1, $t1, $s1 # t1 = 65x
	add $t1, $t1, $s1 # t1 = 66x
	add $t1, $t1, $s1 # t1 = 67x

	add $t2, $t0, $t1 # t2 = t0 + t1
	sll $t2, $t2, 2 # t2 = 4*t2
	add $s2, $zero, $t2 # z = t2
#fim
```

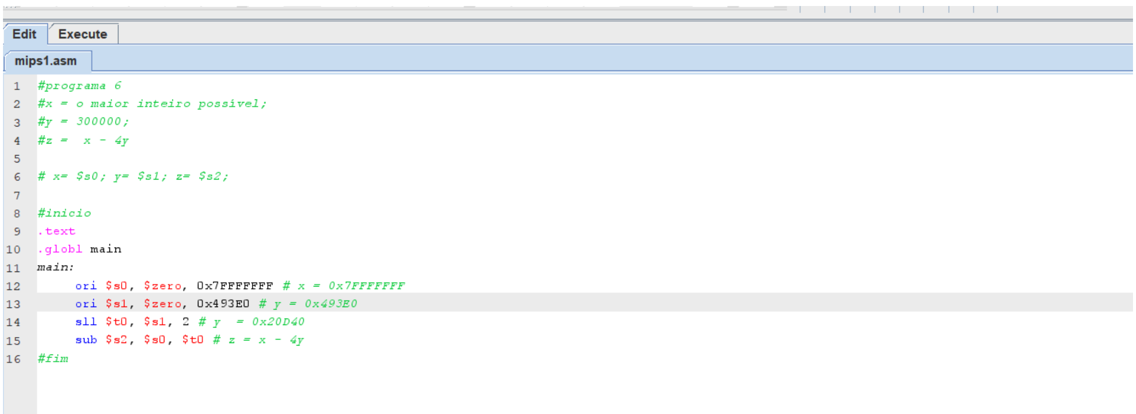


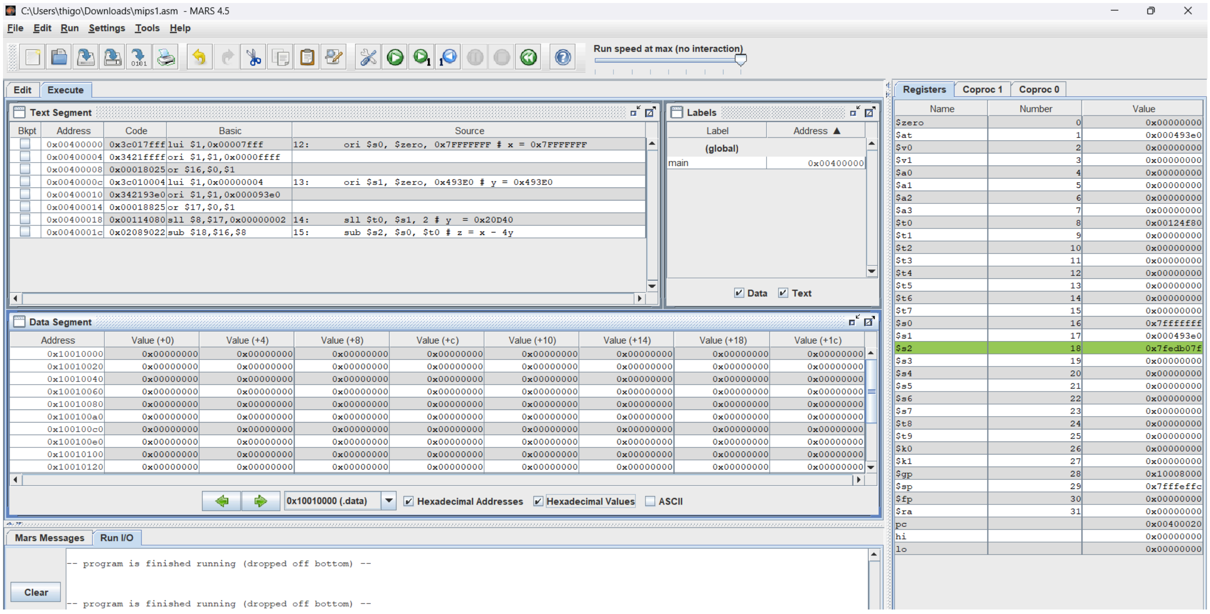
Programa 5:


Edit
Execute
mips1.asm
#programa 5
#x = 100000;
#y = 200000;
#z = x + y;
x= $s0; y= $s1; z= $s2;
#inicio
.text
.globl main
main:
ori $s0, $zero, 0x186A # x = 0x186A
sll $s0, $s0, 4 # x = 0x186A0
ori $s1, $zero, 0x30D4 # y = 0x20D4
sll $s1, $s1, 4 # y = 0x20D40
add $s2, $s0, $s1 # z = x + y
#fim
Line: 16 Column: 31
Show Line Numbers
Mars Messages
Run I/O
-- program is finished running (dropped off bottom) --
Clear
-- program is finished running (dropped off bottom) --

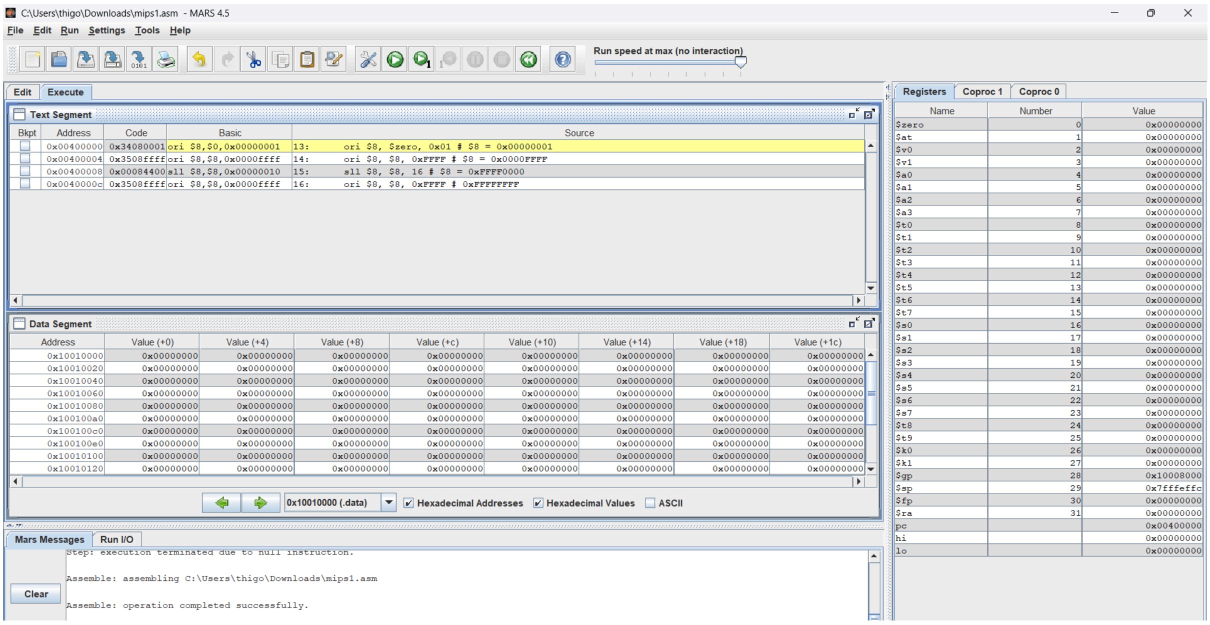

C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit
Execute
Text Segment
Labels
Data Segment
Registers
Coproc 1
Coproc 0
Mars Messages
Run I/O
Clear
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --

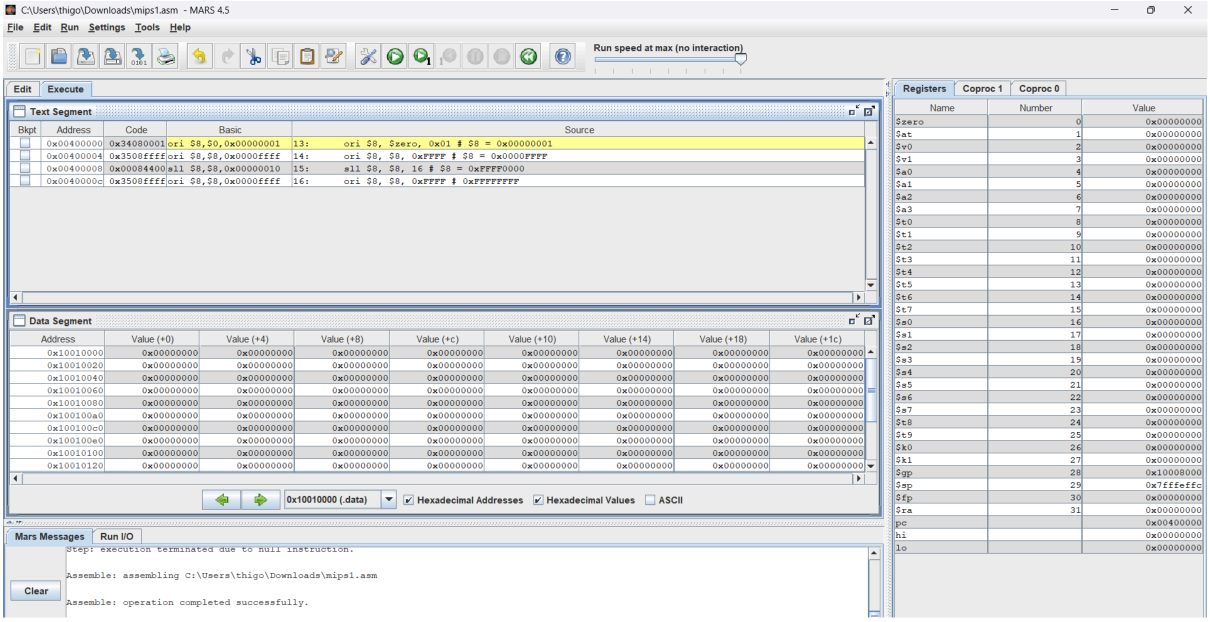
Programa 6:


Edit
Execute
mips1.asm
#programa 6
#x = o maior inteiro possivel;
#y = 300000;
#z = x - 4y
x= $s0; y= $s1; z= $s2;
#inicio
.text
.globl main
main:
ori $s0, $zero, 0x7FFFFFFF # x = 0x7FFFFFFF
ori $s1, $zero, 0x493E0 # y = 0x493E0
sll $t0, $s1, 2 # y = 0x20D40
sub $s2, $s0, $t0 # z = x - 4y
#fim

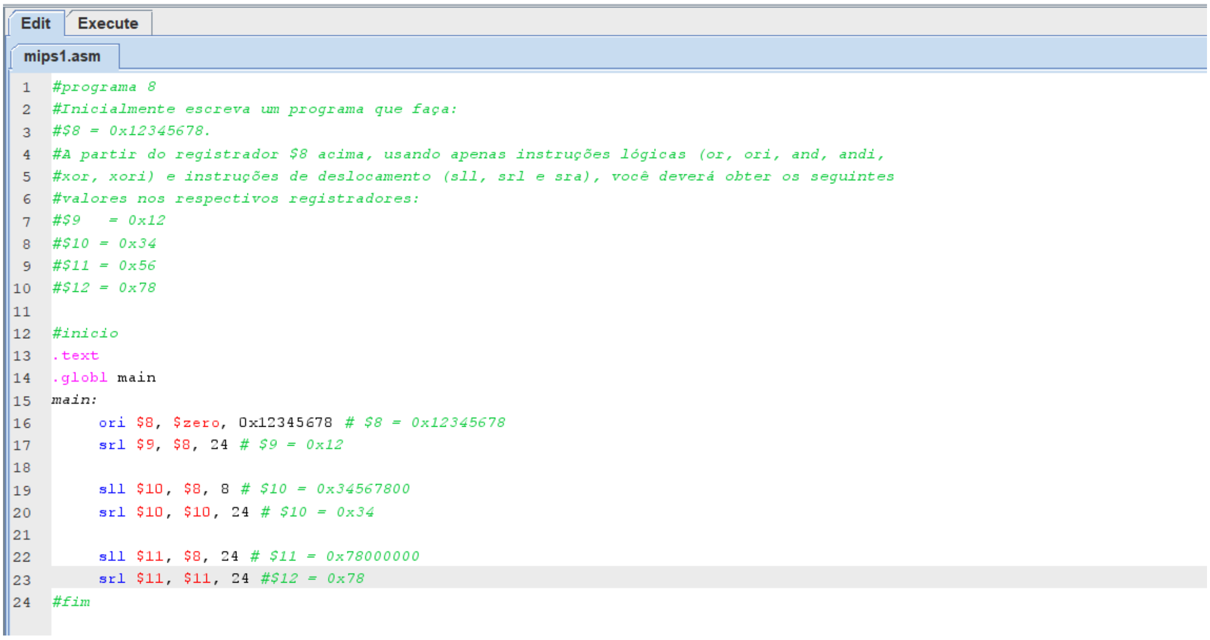

C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
ori $s0, $zero, 0x7FFFFFFF # x = 0x7FFFFFFF
ori $s1, $zero, 0x493E0 # y = 0x493E0
sll $t0, $s1, 2 # y = 0x20D40
sub $s2, $s0, $t0 # z = x - 4y
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --

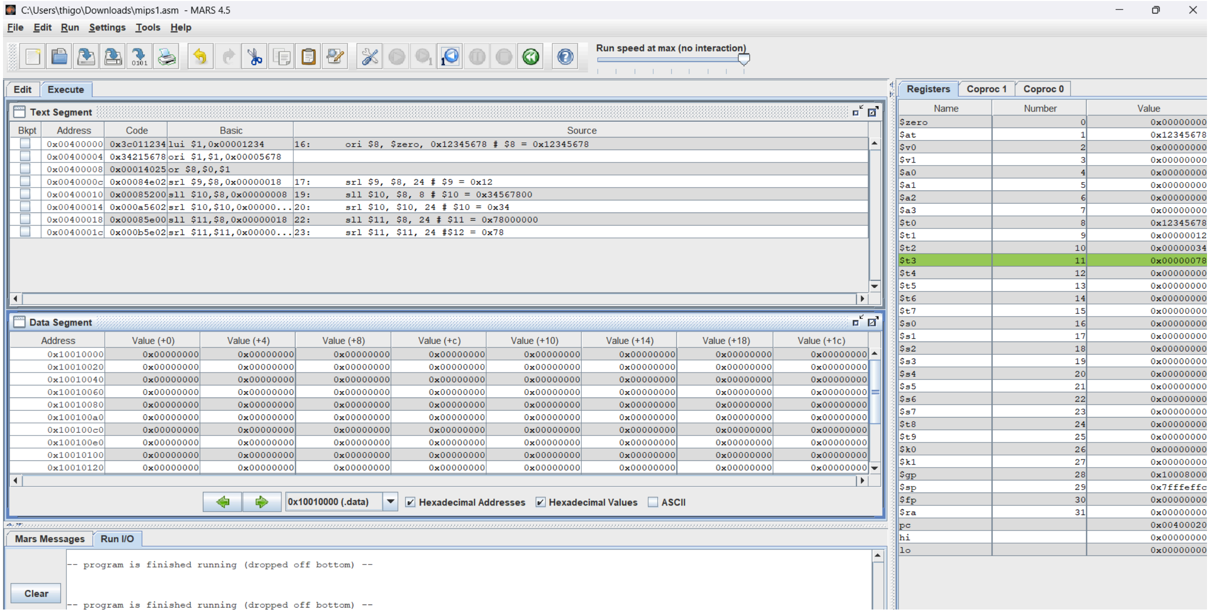
Programa 7:


C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
ori $8, $zero, 0x01 # $8 = 0x00000001
ori $8, $8, 0xFFFF # $8 = 0x0000FFFF
sll $8, $8, 16 # $8 = 0xFFFF0000
ori $8, $8, 0xFFFF # 0xFFFFFFFF
Step: execution terminated due to null instruction.
Assemble: assembling C:\Users\thigo\Downloads\mips1.asm
Assemble: operation completed successfully.


C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x34080001 ori $8,$0,0x00000001 13: ori $8, $zero, 0x01 # $8 = 0x00000001
0x00400004 0x3508ffff ori $8,$8,0x0000ffff 14: ori $8, $8, 0xFFFF # $8 = 0x0000FFFF
0x00400008 0x00084400 sll $8,$8,0x00000010 15: sll $8, $8, 16 # $8 = 0xFFFF0000
0x0040000c 0x3508ffff ori $8,$8,0x0000ffff 16: ori $8, $8, 0xFFFF # 0xFFFFFFFF
Data Segment
Address Value (+0) Value (+4) Value (+8) Value (+c) Value (+10) Value (+14) Value (+18) Value (+1c)
0x10010000 (.data) Hexadecimal Addresses Hexadecimal Values ASCII
Mars Messages Run I/O
Clear
Step: execution terminated due to null instruction.
Assemble: assembling C:\Users\thigo\Downloads\mips1.asm
Assemble: operation completed successfully.
Registers Coproc 1 Coproc 0
Name Number Value
$gp 28 0x10008000
$sp 29 0x7fffeffc
pc 0x00400000

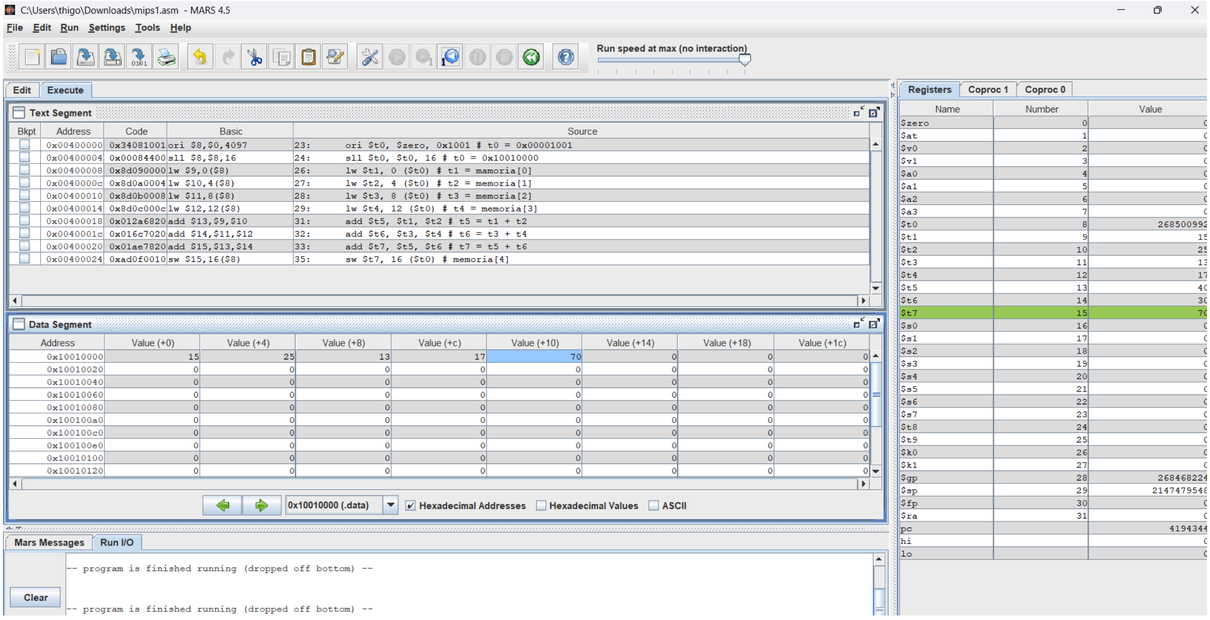
Programa 8:


Edit Execute
mips1.asm
#programa 8
#Inicialmente escreva um programa que faça:
#$8 = 0x12345678.
#A partir do registrador $8 acima, usando apenas instruções lógicas (or, ori, and, andi,
#xor, xori) e instruções de deslocamento (sll, srl e sra), você deverá obter os seguintes
#valores nos respectivos registradores:
#$9 = 0x12
#$10 = 0x34
#$11 = 0x56
#$12 = 0x78
#inicio
.text
.globl main
main:
ori $8, $zero, 0x12345678 # $8 = 0x12345678
srl $9, $8, 24 # $9 = 0x12
sll $10, $8, 8 # $10 = 0x34567800
srl $10, $10, 24 # $10 = 0x34
sll $11, $8, 24 # $11 = 0x78000000
srl $11, $11, 24 #$12 = 0x78
#fim



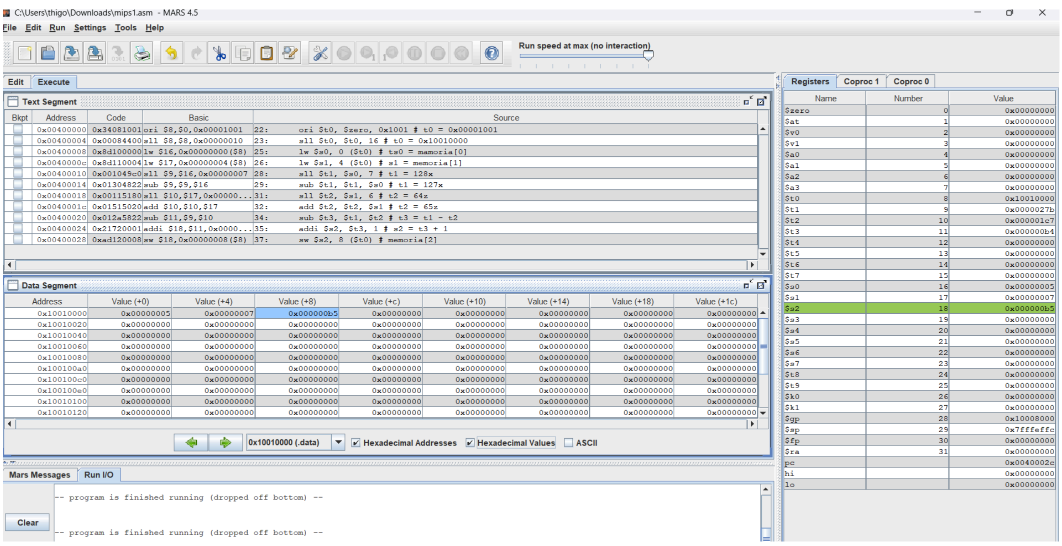
Programa 9:

```
#Escrever um programa que leia todos os números, calcule e substitua o valor da variável soma por este valor.

#inicio
.data
x1: .word 15
x2: .word 25
x3: .word 13
x4: .word 17
soma: .word -1

.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $t1, 0 ($t0) # t1 = mamoria[0]
	lw $t2, 4 ($t0) # t2 = memoria[1]
	lw $t3, 8 ($t0) # t3 = memoria[2]
	lw $t4, 12 ($t0) # t4 = memoria[3]

	add $t5, $t1, $t2 # t5 = t1 + t2
	add $t6, $t3, $t4 # t6 = t3 + t4
	add $t7, $t5, $t6 # t7 = t5 + t6

	sw $t7, 16 ($t0) # memoria[4]
#fim
```



Programa 10:

```
# x= $s0; z= $s1; y= $s2;
#inicio
.data
x: .word 5
z: .word 7
y: .word 0

.text
.globl main
main:
    ori $t0, $zero, 0x1001 # t0 = 0x00001001
    sll $t0, $t0, 16 # t0 = 0x10010000

    lw $s0, 0 ($t0) # ts0 = mamoria[0]
    lw $s1, 4 ($t0) # s1 = memoria[1]

    sll $t1, $s0, 7 # t1 = 128x
    sub $t1, $t1, $s0 # t1 = 127x

    sll $t2, $s1, 6 # t2 = 64z
    add $t2, $t2, $s1 # t2 = 65z

    sub $t3, $t1, $t2 # t3 = t1 - t2
    addi $s2, $t3, 1 # s2 = t3 + 1

    sw $s2, 8 ($t0) # memoria[2]
#fim
```

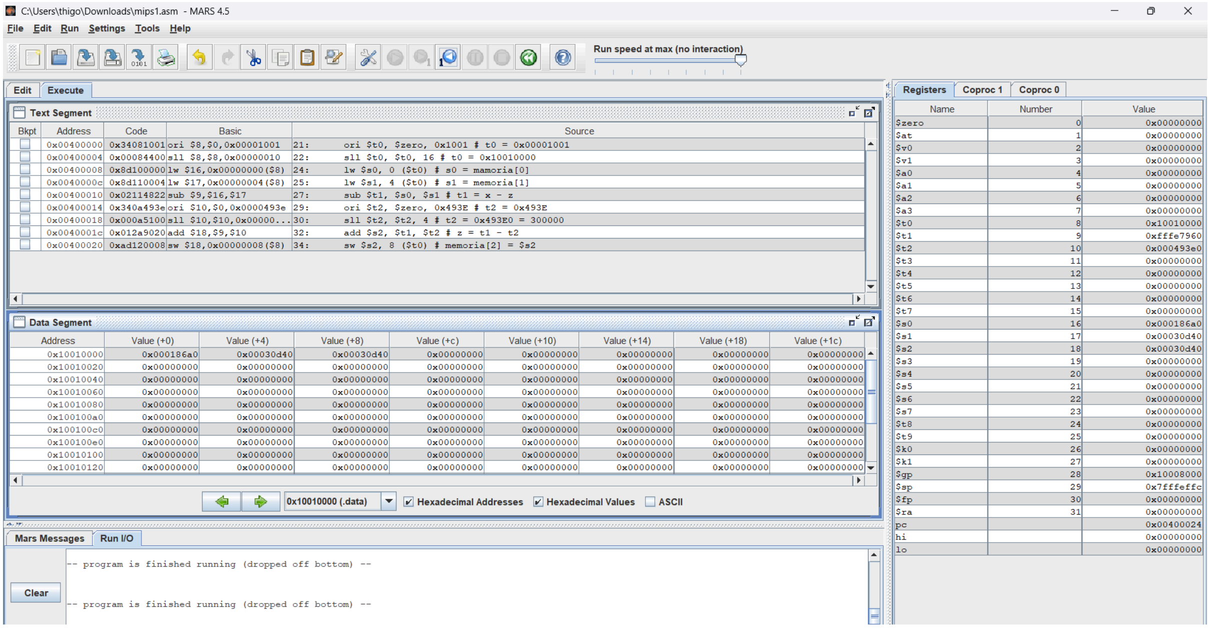


Programa 11:

Edit | Execute

mips1.asm*

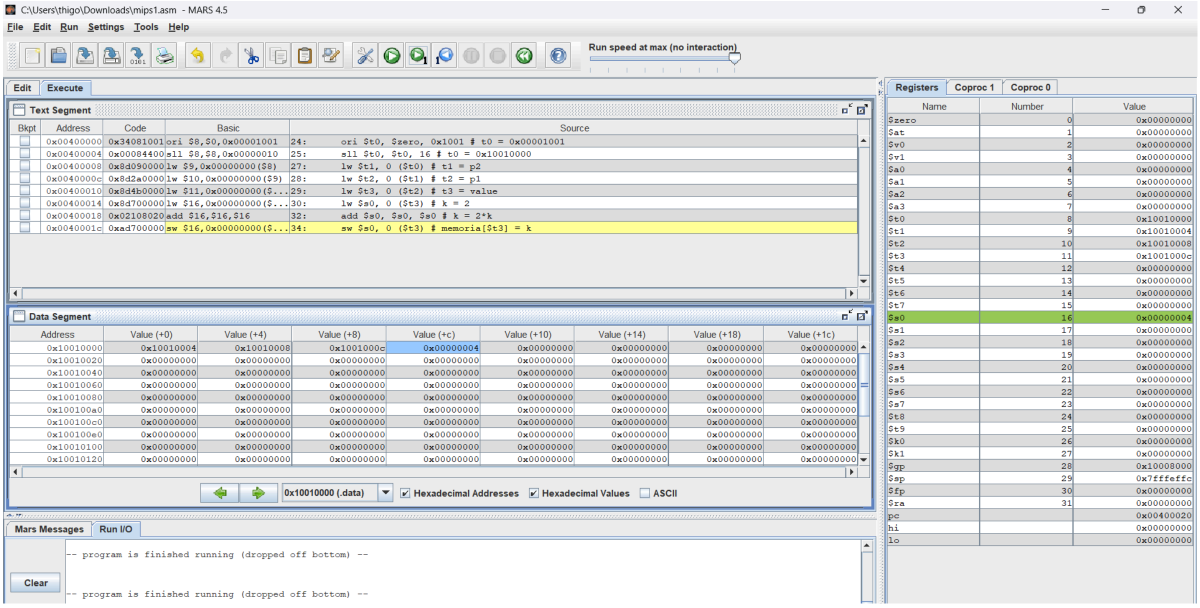
```
#y: .word 0   # esse valor deverá ser sobrescrito após a execução do programa.

# x= $s0; z= $s1; y= $s2;
#inicio
.data
x: .word 100000
z: .word 200000
y: .word 0   # esse valor deverá ser sobrescrito após a execução do programa.

.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $s0, 0 ($t0) # s0 = mamoria[0]
	lw $s1, 4 ($t0) # s1 = memoria[1]

	sub $t1, $s0, $s1 # t1 = x - z

	ori $t2, $zero, 0x493E # t2 = 0x493E
	sll $t2, $t2, 4 # t2 = 0x493E0 = 300000

	add $s2, $t1, $t2 # z = t1 - t2

	sw $s2, 8 ($t0) # memoria[2] = $s2
#fim
```



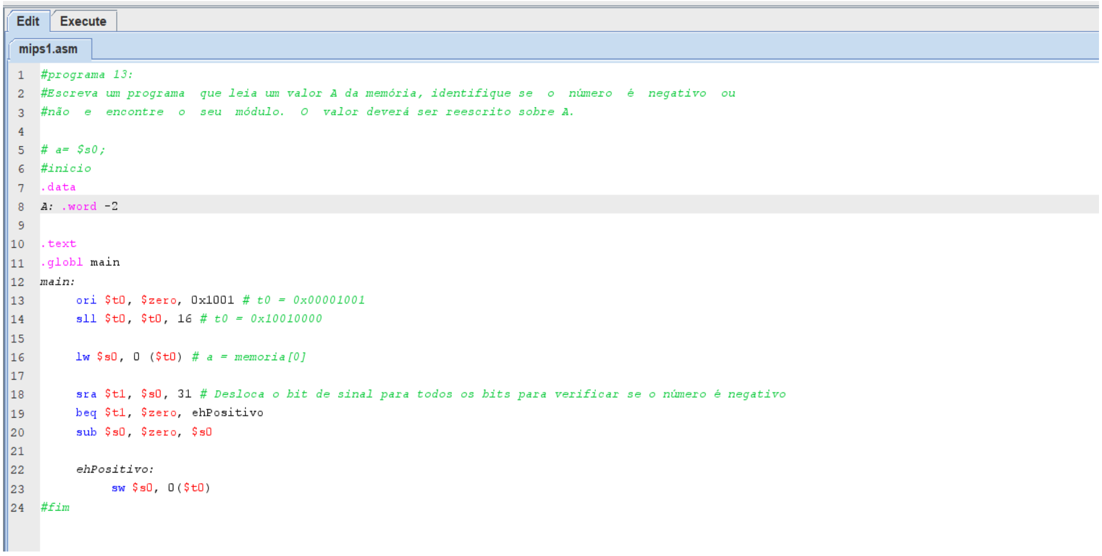
Programa 12:

mips1.asm

```
#k = MEM [ MEM [MEM [ x ] ] ].
#Crie um programa que implemente a estrutura de dados acima, leia o valor de K, o multiplique por
#2 e o reescreva no local correto conhecendo-se apenas o valor de x.

# k= $s0;
#inicio
.data
x: .word p2          # Terceiro ponteiro que aponta para 'p2' (***x)
p2: .word p1          # Segundo ponteiro que aponta para 'p1'
p1: .word value       # Primeiro ponteiro que aponta para 'valor'
value: .word 2

.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $t1, 0 ($t0) # t1 = p2
	lw $t2, 0 ($t1) # t2 = p1
	lw $t3, 0 ($t2) # t3 = value
	lw $s0, 0 ($t3) # k = 2

	add $s0, $s0, $s0 # k = 2*k

	sw $s0, 0 ($t3) # memoria[$t3] = k
#fim
```


C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x34081001 ori $8,$0,0x00001001 24: ori $t0, $zero, 0x1001 # t0 = 0x00001001
0x00400004 0x00084400 sll $8,$8,0x00000010 25: sll $t0, $t0, 16 # t0 = 0x10010000
0x00400008 0x8d090000 lw $9,0x00000000($8) 27: lw $t1, 0 ($t0) # t1 = p2
0x0040000c 0x8d2a0000 lw $10,0x00000000($9) 28: lw $t2, 0 ($t1) # t2 = p1
0x00400010 0x8d4b0000 lw $11,0x00000000($... 29: lw $t3, 0 ($t2) # t3 = value
0x00400014 0x8d700000 lw $16,0x00000000($... 30: lw $s0, 0 ($t3) # k = 2
0x00400018 0x02108020 add $16,$16,$16 32: add $s0, $s0, $s0 # k = 2*k
0x0040001c 0xad700000 sw $16,0x00000000($... 34: sw $s0, 0 ($t3) # memoria[$t3] = k
Data Segment
Registers Coproc 1 Coproc 0
Mars Messages Run I/O
-- program is finished running (dropped off bottom) --
Clear
-- program is finished running (dropped off bottom) --

Programa 13:


Edit Execute
mips1.asm
#programa 13:
#Escreva um programa que leia um valor A da memória, identifique se o número é negativo ou
#não e encontre o seu módulo. O valor deverá ser reescrito sobre A.

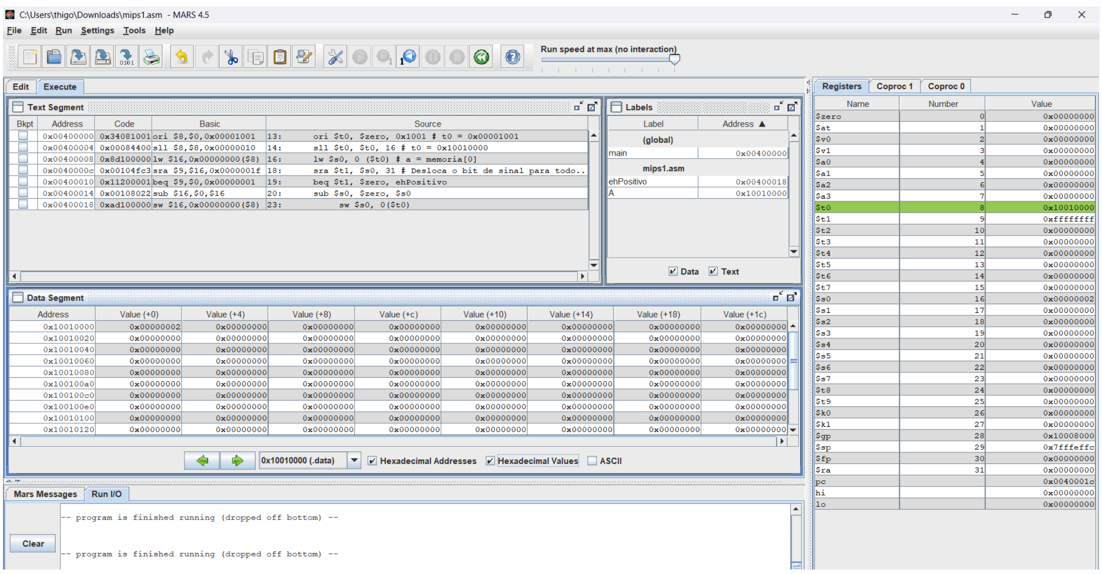
a= $s0;
#inicio
.data
A: .word -2

.text
.globl main
main:
ori $t0, $zero, 0x1001 # t0 = 0x00001001
sll $t0, $t0, 16 # t0 = 0x10010000

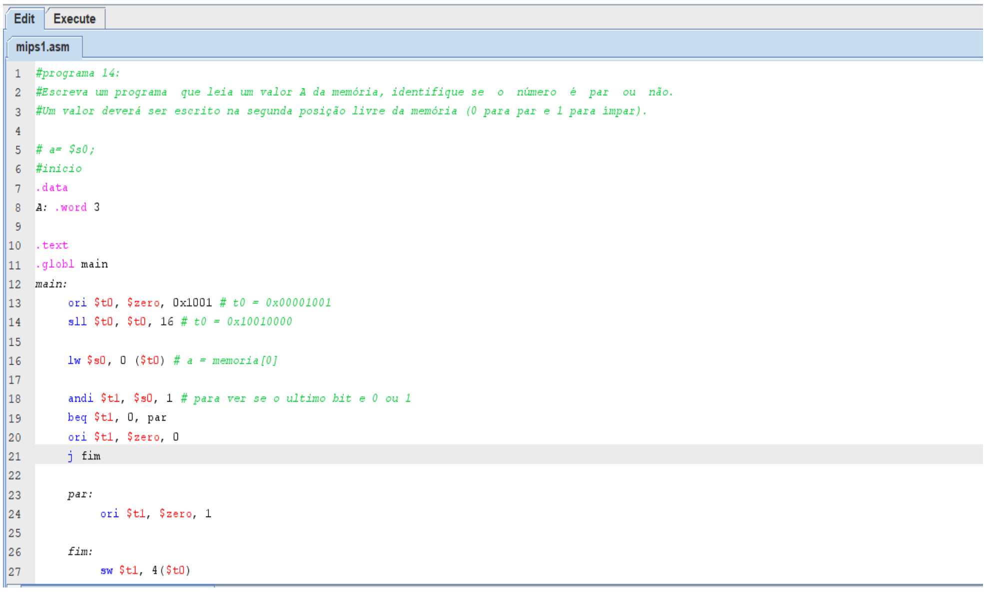
lw $s0, 0 ($t0) # a = memoria[0]

sra $t1, $s0, 31 # Desloca o bit de sinal para todos os bits para verificar se o número é negativo
beq $t1, $zero, ehPositivo
sub $s0, $zero, $s0

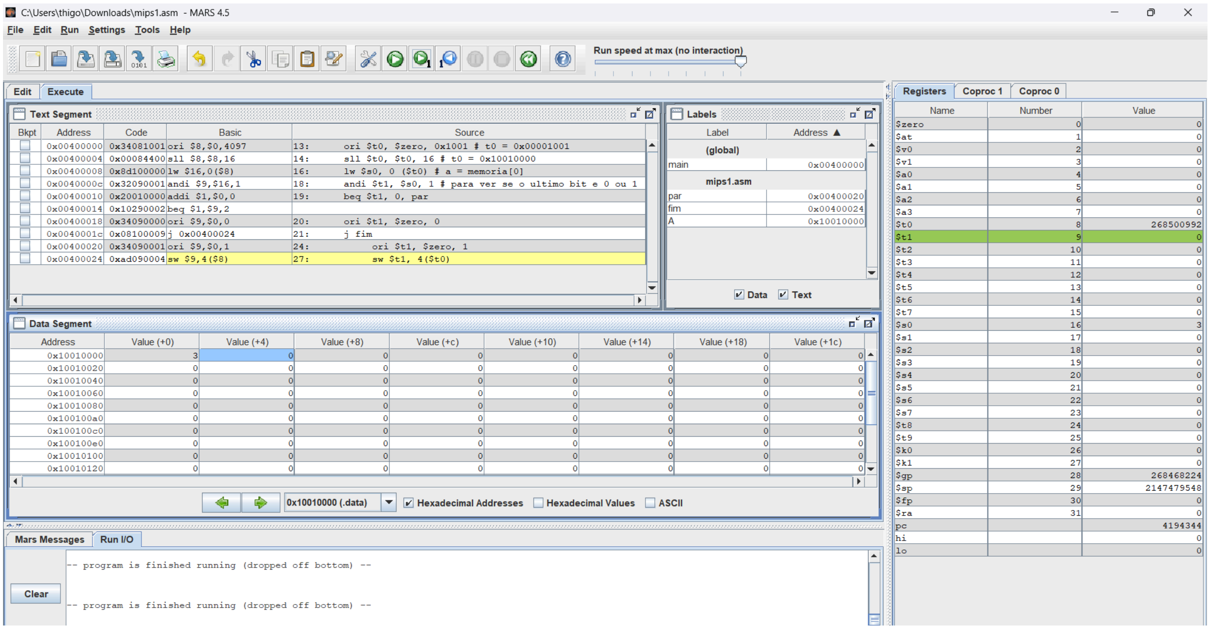
ehPositivo:
sw $s0, 0($t0)
#fim


C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x34081001 ori $8,$0,0x00001001 13: ori $t0, $zero, 0x1001 # t0 = 0x00001001
0x00400004 0x00084400 sll $8,$8,0x00000010 14: sll $t0, $t0, 16 # t0 = 0x10010000
0x00400008 0x8d100000 lw $16,0x00000000($8) 16: lw $s0, 0 ($t0) # a = memoria[0]
0x0040000c 0x00104fc3 sra $9,$16,0x0000001f 18: sra $t1, $s0, 31 # Desloca o bit de sinal para todo..
0x00400010 0x11200001 beq $9,$0,0x00000001 19: beq $t1, $zero, ehPositivo
0x00400014 0x00108022 sub $16,$0,$16 20: sub $s0, $zero, $s0
0x00400018 0xad100000 sw $16,0x00000000($8) 23: sw $s0, 0($t0)
Labels
Label Address
(global)
main 0x00400000
mips1.asm
ehPositivo 0x00400018
A 0x10010000
Data Text
Data Segment
Registers Coproc 1 Coproc 0
Mars Messages Run I/O
-- program is finished running (dropped off bottom) --
Clear
-- program is finished running (dropped off bottom) --

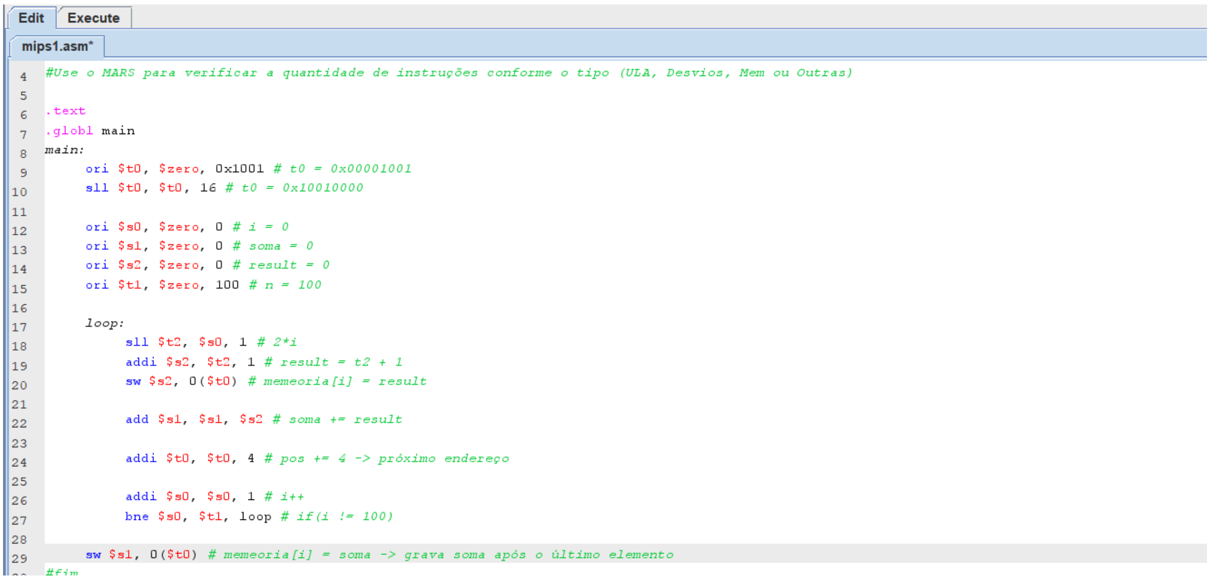
Programa 14:

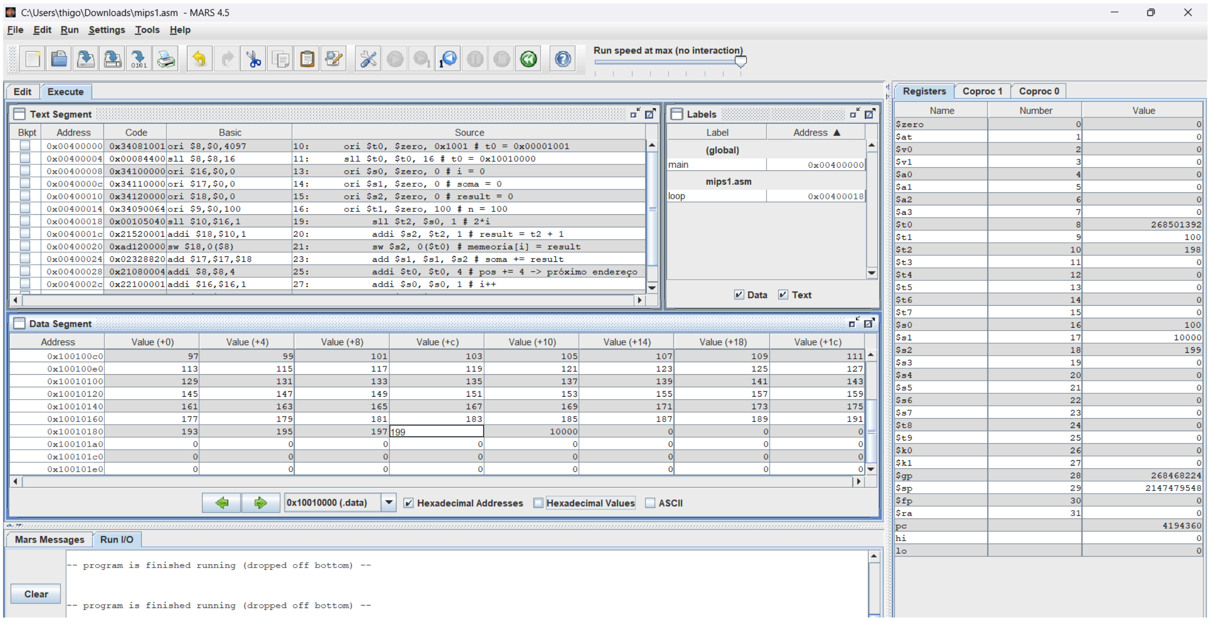


```
#programa 14:
#Escreva um programa  que leia um valor A da memória, identifique se  o  número  é  par  ou  não.
#Um valor deverá ser escrito na segunda posição livre da memória (0 para par e 1 para impar).

# a= $s0;
#inicio
.data
A: .word 3

.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $s0, 0 ($t0) # a = memoria[0]

	andi $t1, $s0, 1 # para ver se o ultimo bit e 0 ou 1
	beq $t1, 0, par
	ori $t1, $zero, 0
	j fim

	par:
		ori $t1, $zero, 1

	fim:
		sw $t1, 4($t0)
```



Programa 15:


Edit
Execute
mips1.asm*
#Use o MARS para verificar a quantidade de instruções conforme o tipo (ULA, Desvios, Mem ou Outras)
.text
.globl main
main:
ori $t0, $zero, 0x1001 # t0 = 0x00001001
sll $t0, $t0, 16 # t0 = 0x10010000
ori $s0, $zero, 0 # i = 0
ori $s1, $zero, 0 # soma = 0
ori $s2, $zero, 0 # result = 0
ori $t1, $zero, 100 # n = 100
loop:
sll $t2, $s0, 1 # 2*i
addi $s2, $t2, 1 # result = t2 + 1
sw $s2, 0($t0) # memeoria[i] = result
add $s1, $s1, $s2 # soma += result
addi $t0, $t0, 4 # pos += 4 -> próximo endereço
addi $s0, $s0, 1 # i++
bne $s0, $t1, loop # if(i != 100)
sw $s1, 0($t0) # memeoria[i] = soma -> grava soma após o último elemento

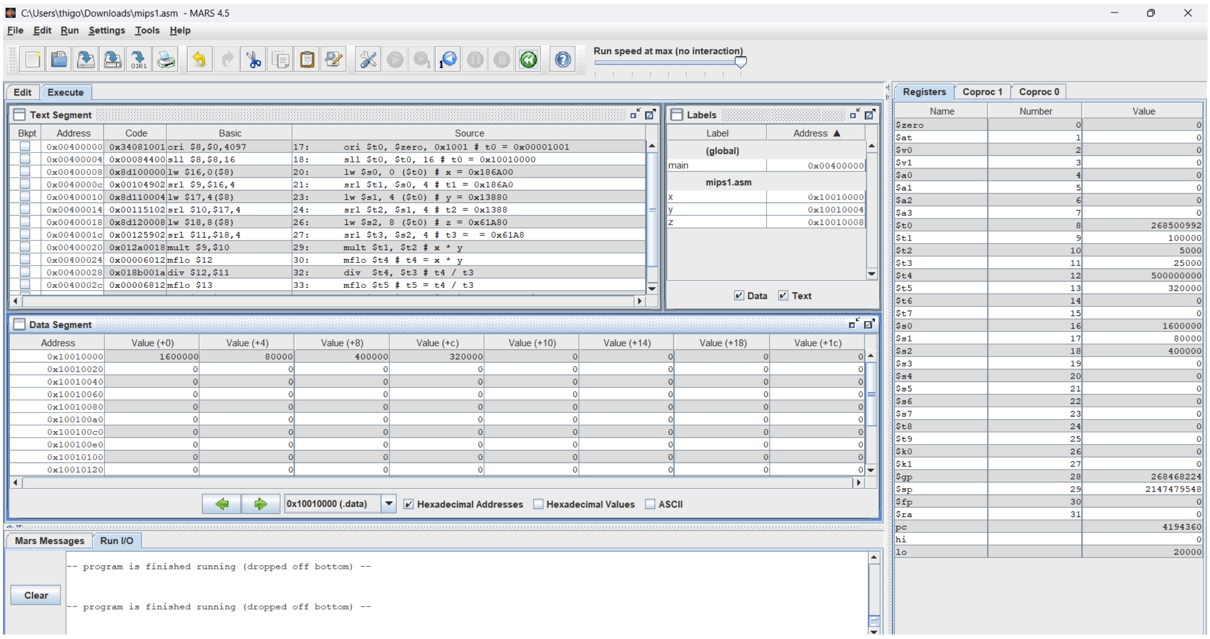

C:\Users\thigo\Downloads\mips1.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Text Segment
Labels
Data Segment
Registers Coproc 1 Coproc 0
Mars Messages Run I/O
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --

Programa 16:

Edit | Execute

mips2.asm | mips1.asm

```
.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $s0, 0 ($t0) # x = 0x186A00
	srl $t1, $s0, 4 # t1 = 0x186A0

	lw $s1, 4 ($t0) # y = 0x13880
	srl $t2, $s1, 4 # t2 = 0x1388

	lw $s2, 8 ($t0) # z = 0x61A80
	srl $t3, $s2, 4 # t3 =  = 0x61A8

	mult $t1, $t2 # x * y
	mflo $t4 # t4 = x * y

	div  $t4, $t3 # t4 / t3
	mflo $t5 # t5 = t4 / t3

	sll $t5, $t5, 4 # vai mover os 4bits para a esquerda que eu tirei
	sw $t5, 12 ($t0) # mempria[3] = t5

#fim
```

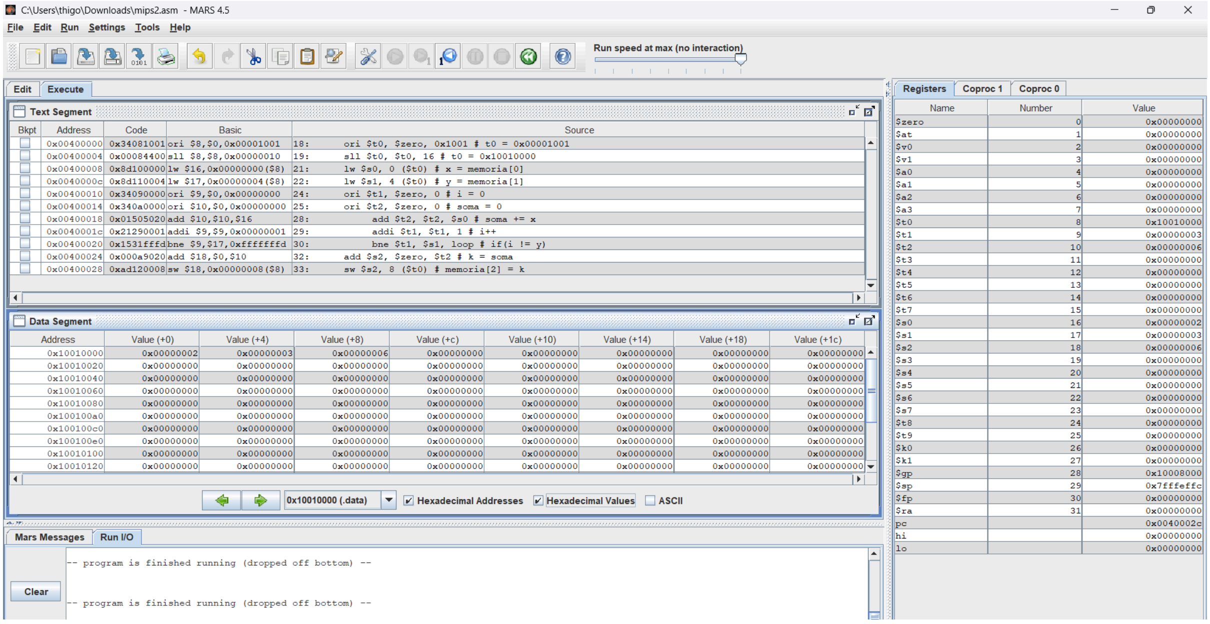


Programa 17:

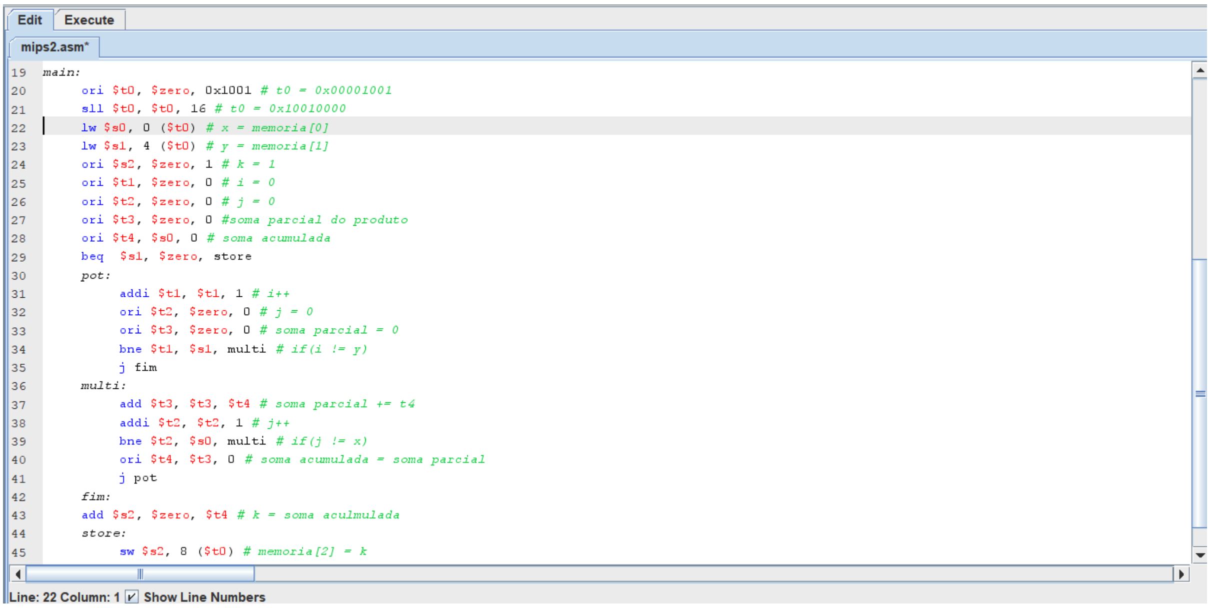
Edit | Execute

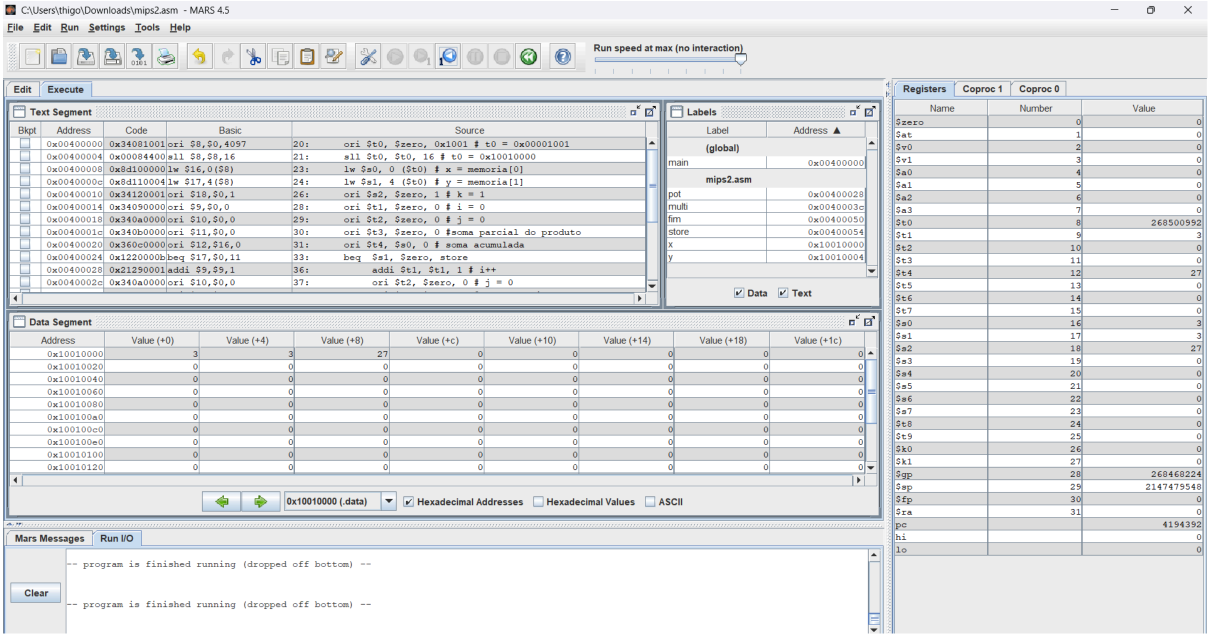
mips1.asm | mips2.asm

```
# x= $s0; y= $s1; k= $s2

.data
	x: .word 2
	y: .word 3

#inicio
.text
.globl main
main:
	ori $t0, $zero, 0x1001 # t0 = 0x00001001
	sll $t0, $t0, 16 # t0 = 0x10010000

	lw $s0, 0 ($t0) # x = memoria[0]
	lw $s1, 4 ($t0) # y = memoria[1]

	ori $t1, $zero, 0 # i = 0
	ori $t2, $zero, 0 # soma = 0

	loop:
		add $t2, $t2, $s0 # soma += x
		addi $t1, $t1, 1 # i++
		bne $t1, $s1, loop # if(i != y)

	add $s2, $zero, $t2 # k = soma
	sw $s2, 8 ($t0) # memoria[2] = k
#fim
```


C:\Users\thigo\Downloads\mips2.asm - MARS 4.5
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Edit Execute
Text Segment
Data Segment
Registers Coproc 1 Coproc 0
Mars Messages Run I/O
Clear
-- program is finished running (dropped off bottom) --
-- program is finished running (dropped off bottom) --

Programa 18:


Edit Execute
mips2.asm*
main:
ori $t0, $zero, 0x1001 # t0 = 0x00001001
sll $t0, $t0, 16 # t0 = 0x10010000
lw $s0, 0 ($t0) # x = memoria[0]
lw $s1, 4 ($t0) # y = memoria[1]
ori $s2, $zero, 1 # k = 1
ori $t1, $zero, 0 # i = 0
ori $t2, $zero, 0 # j = 0
ori $t3, $zero, 0 #soma parcial do produto
ori $t4, $s0, 0 # soma acumulada
beq $s1, $zero, store
pot:
addi $t1, $t1, 1 # i++
ori $t2, $zero, 0 # j = 0
ori $t3, $zero, 0 # soma parcial = 0
bne $t1, $s1, multi # if(i != y)
j fim
multi:
add $t3, $t3, $t4 # soma parcial += t4
addi $t2, $t2, 1 # j++
bne $t2, $s0, multi # if(j != x)
ori $t4, $t3, 0 # soma acumulada = soma parcial
j pot
fim:
add $s2, $zero, $t4 # k = soma aculmulada
store:
sw $s2, 8 ($t0) # memoria[2] = k
Line: 22 Column: 1 Show Line Numbers



Perguntas:

1. Se tivermos 2 inteiros, cada um com 32 bits, quantos bits podemos esperar para o produto?

C. 64

2. Quais os registradores que armazenam os resultados na multiplicação?

B. hi e lo

3. Qual a operação usada para multiplicar inteiros em comp. de dois?

A. mult

4. Qual instrução move os bits menos significativos da multiplicação para o reg. 8?

C. mflo $8

5. Se tivermos dois inteiros, cada um com 32 bits, quantos bits deveremos estar preparados para receber no quociente?

B. 32

6. Após a instrução div, qual registrador possui o quociente?

A. lo

7. Qual a inst. Usada para dividir dois inteiros em comp. de dois?

D. div

8. Faça um arithmetic shift right de dois no seguinte padrão de bits: 1001 1011

A. 1110 0110

9. Qual o efeito de um arithmetic shift right de uma posição?

C. Se o inteiro for unsigned, o shift pode ocasionar um valor errado. Se o inteiro for signed, o shift o divide por 2.

10. Qual sequencia de instruções avalia 3x+7, onde x é iniciado no reg. $8 e o resultado armazenado em $9?

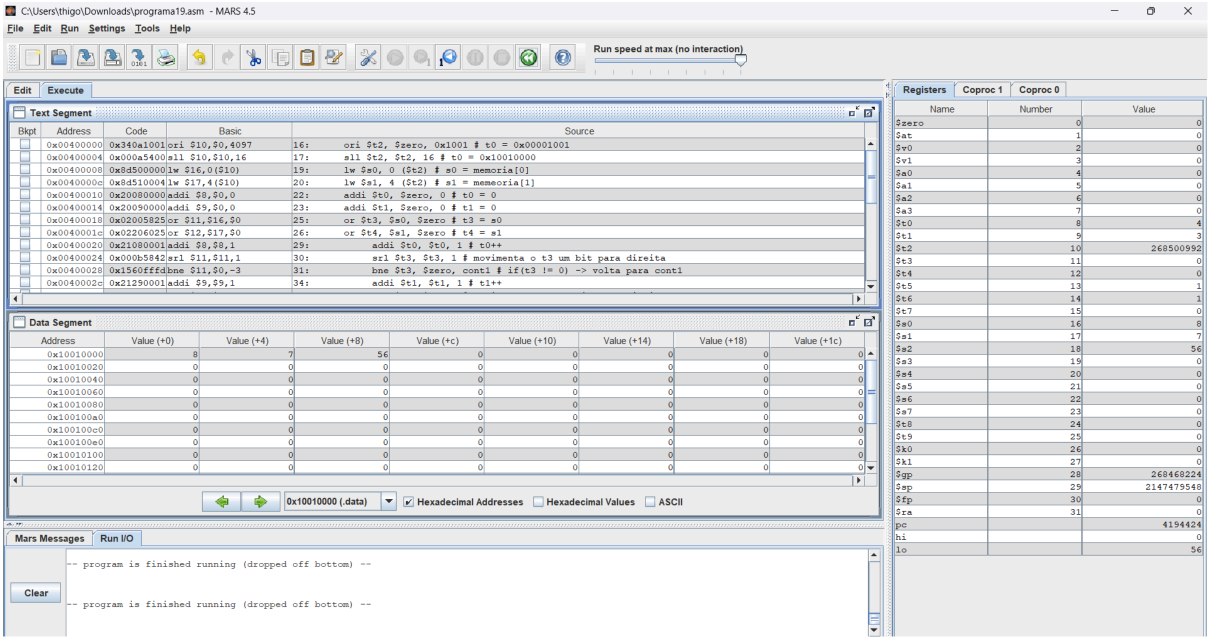
A.
ori $3,$0,3
mult $8,$3
mflo $9
addi $9,$9,7

Implemente os programas a seguir no MIPS/MARS:

Programa 19:

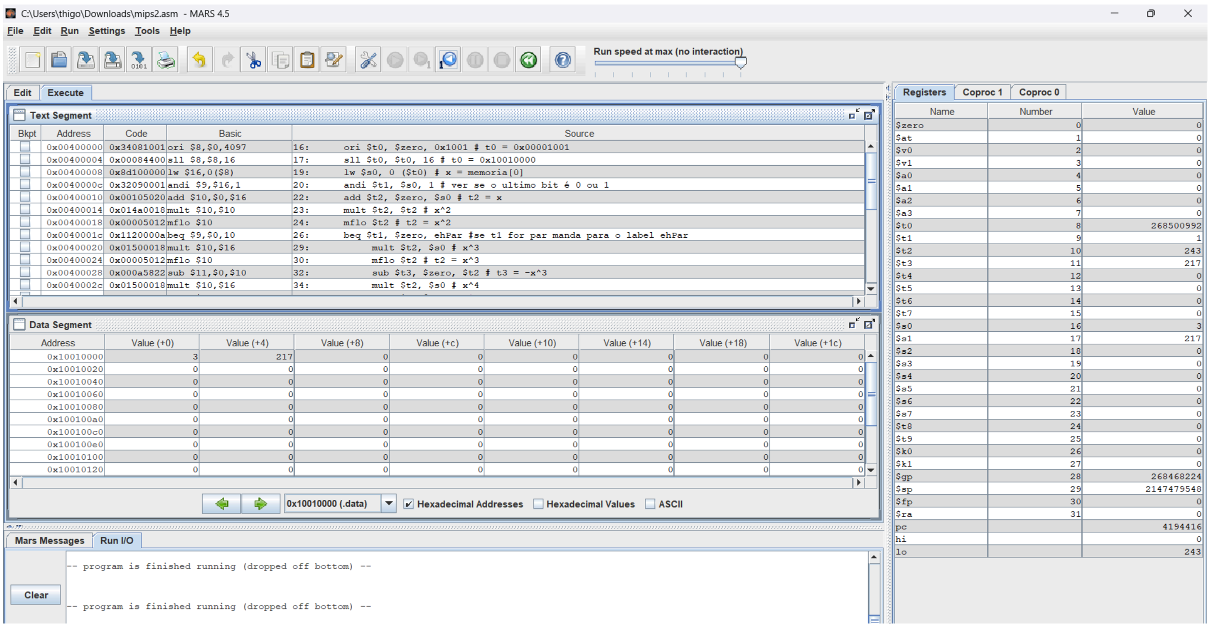
```
.text
.globl main
main:
	ori $t2, $zero, 0x1001 # t0 = 0x00001001
	sll $t2, $t2, 16 # t0 = 0x10010000

	lw $s0, 0 ($t2) # s0 = memoria[0]
	lw $s1, 4 ($t2) # s1 = memeoria[1]

	addi $t0, $zero, 0 # t0 = 0
	addi $t1, $zero, 0 # t1 = 0

	or $t3, $s0, $zero # t3 = s0
	or $t4, $s1, $zero # t4 = s1

	cont1:
		addi $t0, $t0, 1 # t0++
		srl $t3, $t3, 1 # movimenta o t3 um bit para direita
		bne $t3, $zero, cont1 # if(t3 != 0) -> volta para cont1

	cont2:
		addi $t1, $t1, 1 # t1++
		srl $t4, $t4, 1 # movimenta o t3 um bit para direita
		bne $t4, $zero, cont2 # if(t4 != 0) -> volta para cont2

	check:
		slti $t5, $t0, 32 # se t0 < 32 -> t5 = 1
```

```
        slti $t5, $t0, 32 # se t0 < 32 -> t5 = 1
        slti $t6, $t1, 32 # se t1 < 32 -> t6 = 1
        add $t7, $t5, $t6 # t7 = t5 + t6

        slti $t7, $t7, 2 # se t7 < 2, então t7 = 1
        beq $t7, $zero, multi2 # if(t7 == 0) -> vai para multi2

    multi1:
        mult $s0, $s1 # s0 * s1
        mflo $s2 # s2 recebe os bits menos significativos de resultado
        mfhi $s3 # s3 recebe os bits mais significativos de resultado
        j store1

    multi2:
        mult $s0, $s1 # s0 * s1
        mflo $s2 # s2 recebe os bits menos significativos de resultado
        j store2

    store1:
        sw $s2, 8 ($t2) # memoria[2] = s2
        sw $s3, 12 ($t2) # memoria[3] = s3
        j fim

    store2:
        sw $s2, 8 ($t2) # memoria[2] = s2

    fim:
```



Programa 20:

```
# x= $s0; y= $s1;
.data
      x: .word 3

#inicio
.text
.globl main
main:
      ori $t0, $zero, 0x1001 # t0 = 0x00001001
      sll $t0, $t0, 16 # t0 = 0x10010000
      lw $s0, 0 ($t0) # x = memoria[0]
      andi $t1, $s0, 1 # ver se o ultimo bit é 0 ou 1
      add $t2, $zero, $s0 # t2 = x
      mult $t2, $t2 # x^2
      mflo $t2 # t2 = x^2
      beq $t1, $zero, ehPar #se t1 for par manda para o label ehPar
      ehImpar:
            mult $t2, $s0 # x^3
            mflo $t2 # t2 = x^3
            sub $t3, $zero, $t2 # t3 = -x^3
            mult $t2, $s0 # x^4
            mflo $t2 # t2 = x^4
            mult $t2, $s0 # x^5
            mflo $t2 # t2 = x^5
            add $t3, $t3, $t2 #
            add $t3, $t3, 1 # t3 += 1
            j fim
      ehPar:
            add $t3, $t2, $t2 # t3 = 2*t2
            sub $t3, $zero, $t3 # t3 = -2*t2
            mult $t2, $s0 # x^3
            mflo $t2 # t2 = x^3
            add $t3, $t2, $t3 # t3 += t2
            mult $t2, $s0 # x^4
            mflo $t2 # t2 = x^4
            add $t3, $t2, $t3 # t3 += t2
      fim:
            add $s1, $zero, $t3 # y = t3
            sw $s1, 4 ($t0) # memoria[1] = y
#fim
```



Programa 21:

mips2.asm*

```
# x= $s0; y= $s1;
.data
    x: .word -2

#inicio
.text
.globl main
main:
    ori $t0, $zero, 0x1001 # t0 = 0x00001001
    sll $t0, $t0, 16 # t0 = 0x10010000
    lw $s0, 0 ($t0) # x = memoria[0]
    andi $t1, $s0, 1 # ver se o ultimo bit é 0 ou 1
    add $t2, $zero, $s0 # t2 = x
    mult $t2, $t2 # x^2
    mflo $t2 # t2 = x^2
    mult $t2, $s0 # x^3
    mflo $t2 # t2 = x^3
    slt $t1, $zero, $s0 # t1 = (0 < s0) ? 1 : 0
    beq $t1, 1, maior # if(t1 == 1) -> manda para o label 'maior'
    menorOuIgual:
        subi $t3, $zero, 1 # t3 = -1
        mult $t2, $s0 # x^4
        mflo $t2 # t2 = x^4
        add $t3, $t3, $t2 # t3 += t2
        j fim
    maior:
        addi $t3, $t2, 1 # t3 = x^3 +1

    fim:
        add $s1, $zero, $t3 # y = t3
        sw $s1, 4 ($t0) # memoria[1] = y
#fim
```